JavaScript Foundations Complete Blueprint

CHALLENGES 1 TO 16 REFERENCE LOG

Challenge 1: Basic Variables & Types

Declaring basic data primitives to store information in memory.

```
let username = "Lawrence";
const activeStatus = true;
let accountBalance = 500;
```

Challenge 2: Basic Mathematics & Variables

Using arithmetic operations to update existing memory states.

```
let points = 10;
points = points + 5; // Updated to 15
points += 2;        // Shortcut syntax to 17
```

Challenge 3: Simple String Concatenation

Combining static text values with dynamic structural variables.

```
let role = "Developer";
let message = "Welcome back, " + username + " (" + role + ")";
```

Challenge 4: Template Literals

Modern syntax backticks for clean inline string variable interpolation.

```
let cleanMessage = `Welcome back, ${username}! Your role is: ${role}.`;
```

Challenge 5: Introduction to Arrays

Declaring zero-indexed structured collections to store sequence groups.

```
let techStack = ["React", "Node.js", "MongoDB"];
console.log(techStack[0]); // Accesses "React"
```

Challenge 6: Array Manipulation (.push & .pop)

Adding and removing items dynamically from the end of collections.

```
let tools = ["Git", "VS Code"];
tools.push("Vite"); // Adds to end
tools.pop();        // Removes last element ("Vite")
```

Challenge 7: Introduction to Objects

Storing structured information using explicit key-value properties.

```
let project = {
  title: "RetailSync-POS",
  version: "1.0.0",
  isDeployed: false
};
console.log(project.title); // Dot notation extraction
```

Challenge 8: Basic For Loops

Iterating dynamically through incremental counting intervals.

```
for (let i = 0; i < 5; i++) {
  console.log(`Current index counter: ${i}`);
}
```

Challenge 9: Iterating Through Arrays

Combining counting parameters with index accessors to step over arrays.

```
let items = ["Laptop", "Mouse", "Keyboard"];
for (let i = 0; i < items.length; i++) {
  console.log(`Processing: ${items[i]}`);
}
```

Challenge 10: The Accumulator Pattern

Tracking, maintaining, and updating running computations during iterations.

```
let cart = [
  { item: 'Laptop', price: 1000 },
  { item: 'Mouse', price: 50 }
];
let totalPrice = 0;
for (let i = 0; i < cart.length; i++) {
  totalPrice += cart[i].price;
}
console.log(totalPrice); // 1050
```

Challenge 11: The Inventory Count (Loop + Condition)

Filtering items within a structural sequence using targeted equality checks.

```
let warehouse = [
  { product: "Laptop", category: "electronics", quantity: 5 },
  { product: "Desk Chair", category: "furniture", quantity: 12 }
];
let totalElectronics = 0;
for (let i = 0; i < warehouse.length; i++) {
  if (warehouse[i].category === 'electronics') {
    totalElectronics += warehouse[i].quantity;
  }
}
console.log(totalElectronics); // 5
```

Challenge 12: Basic Function Blueprint

Packaging code fragments inside reusable structural blocks with parameters.

```
function multiplyByTwo(num) {
  return num * 2;
}
console.log(multiplyByTwo(4)); // 8
```

Challenge 13: Functions with Internal Logic

Embedding comparative analysis blocks inside custom operations.

```
function checkAge(age) {
  if (age >= 18) {
    return "Allowed";
  } else {
    return "Not Allowed";
  }
}
console.log(checkAge(18)); // "Allowed"
```

Challenge 14: Processing Input Arrays inside Functions

Passing structured arrays dynamically as context args inside operations.

```
function findMax(numberArray) {
  let max = numberArray[0];
  for (let i = 0; i < numberArray.length; i++) {
    if (numberArray[i] > max) {
      max = numberArray[i];
    }
  }
  return max;
}
console.log(findMax([2, 8, 4, 6])); // 8
```

Challenge 15: Array Transformation (The Capturing Rule)

Transforming source data items and appending them instantly into a new bucket.

```
function convertToPercentages(decimalArray) {
  let result = [];
  for (let i = 0; i < decimalArray.length; i++) {
    result.push(decimalArray[i] * 100); // Evaluates and captures
  }
  return result;
}
console.log(convertToPercentages([0.15, 0.5])); // [15, 50]
```

Challenge 16: End-to-End Composite System

Combining functions, structural array iteration, object extraction, and flag filters.

```
function getNotificationEmails(userArray) {
  let getEmail = [];
  for (let i = 0; i < userArray.length; i++) {
    if (userArray[i].hasNotifications === true) {
      getEmail.push(userArray[i].email);
    }
  }
  return getEmail;
}
const list = [
  { name: 'Alice', email: 'alice@test.com', hasNotifications: true },
  { name: 'Bob', email: 'bob@test.com', hasNotifications: false }
];
console.log(getNotificationEmails(list)); // ['alice@test.com']
```